

SR. NO.	DATE	TITLE	PAGE NO.	TEACHER'S SIGN
		Micro Project - 4 people - difficult viva		
		8 →		
		1. Intro stages		
		2. - - - - - ambiguities with NLP		
		3. - - - - -	1 example	
		4. - - - - - applications	2 example	
		5. - - - - - levels	- - -	
		6. - - - - - stages	- - -	
		7. - - - - - challenges	- - -	
		8. Knowledge in NLP		

CIA-1 (12/7) 10m

- 4m maths Q. TTR; relative
3m ambiguity scenarios (20s)
3m preprocessing
identify the application,
which stages will be applied

MSF

Q1 (8m) 4/5 2m each co 1, 2, 3

Q2 (12m) 3/4 4m each co 2

Q3 (10m) 2/3 5m each co 2

manipulating

- formulas
- calc.
- final answer

- time management

- <length>
- a) compute perplexities (bigram)
- b) Laplace smoothing (prob.)
- c) TF-IDF (don't do for whole table, do word specific only)
- a) initial transition prob. (tagging) (HMM)
- b) CRF
- c) Viterbi (show state graph)
- d) CFG, left-most derivation + parse tree
- applications
- text to speech
 - autocorrect
 - caption reader
 - ~~scan~~ scanner
- vector final answer

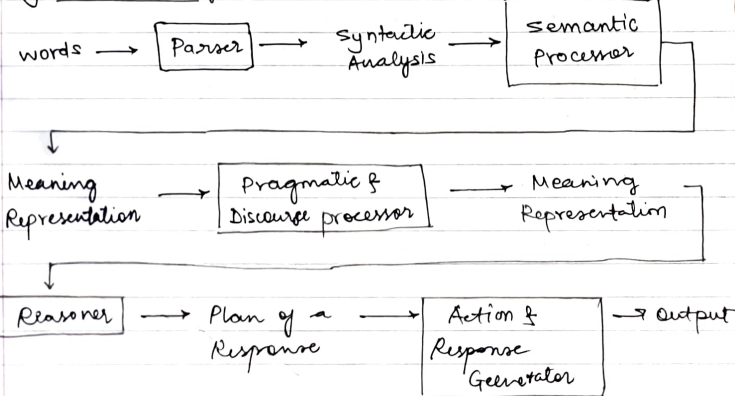
Module 1: Introduction to NLP

- NLP: the branch of AI that enables machines to read, understand & generate human language.

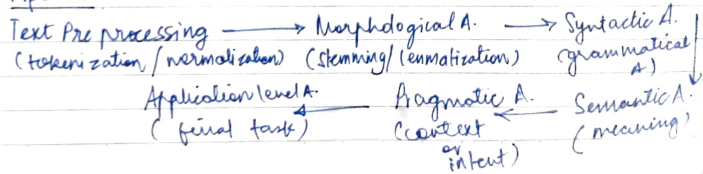
2 components:

- i) NLU: Natural Language Understanding
comp tries to understand human knowledge.
ex: Apple launched a new phone. NLU should understand that Apple = company, not fruit
- ii) NLG: Natural Language Generation
comp generates human-readable language
ex: after understanding "Where is my order?" the chatbot replies "Your order will arrive tomorrow"

- generic NLP system



- Pipeline



0. Preprocessing

- Stages of NLP

morphological analysis

1. Text Analysis: Processing & word level analysis
structure & formation of words (stemming, lemmatization)

2. Syntactic analysis & sequence modeling
grammar & sentence structure (tagging, parsing)

3. Semantic analysis & word representations

tries to determine the meaning
of words & sentences

4. Discourse analysis

5. Pragmatics analysis: context, speaker intuition,
world knowledge

knowledge

how to run, in which
order

- Levels vs Stages

Level (theory)	Stage	Algos / Tools used.
Phonological	1. Preprocessing	
Morphological	2. Morphological P.	
Lexical	3. Lexical / Statistical modelling	
Syntactic	4. Syntactic P.	
Semantic	5. Semantic P.	
Discourse & Pragmatic	6. Discourse & Pragmatic P.	
-- (application)	7. Application	

- Ambiguities & its types:

1. Lexical (or morphological): ambiguity of single word
one word has multiple meanings / grammatical categories

ex: i) The man leaves the bank.

The boat is docked at the river bank.

ii) I have a lot of books.

He didn't book bus tickets, like I asked.

⇒ Part of speech disambiguation - specify if its verb, noun etc.

ex: dog → noun (animal)

dog → verb (to pursue)

⇒ word sense disambiguation - needs word relationships in a context

ex: dog → as animal

dog → as a very detectable person.

2. Syntactic ambiguity / Structural ambiguity
multiple valid sentence structures (parse trees)

solution: parsing / dependency parsing

ex: 'I saw the boy with a telescope'.

interpretations:

1. I used a telescope to see a boy.
2. I saw a boy with who had a telescope.

3.

Semantic ambiguity:

sentence can have multiple meanings even after its grammatically structure is understood.

ex. i) John gave book to Mary.

i) Visiting aunts can be nuisance.

↳ but if it means aunts who are visiting or
as visiting aunts.

ii) "I deposited money in the bank."

↳ it could mean a river bank
or a financial institution.

ii) "Aman loves his mother & Sam does too."

↳ Sam loves Aman's mother

or Sam loves his ^{own} mother.

4. Anaphoric

check the intention of the speaker

a phrase / word refers to something previously mentioned, but there is more than one possibility
multiple possible meanings of he/she/it/they/him/her

ex: Margaret Lam to Shyam that he won the compen.

↳ Ram won

or Shyam won.

5. Pragmatic Ambiguity: < most diff to resolve >

when we are able to tell

check the intention of the speaker.

ex: "can you open the window?"

↳ literal meaning - are you physically able to open the window?

↳ actual interpretation - Please open the window.

- Challenges in NLP

1. Data & availability: large amt of data from multiple sources - diff to handle

2. availability of N. Languages - diff

3. unstructured data

4. Ambiguity

5. Homophones

6. Word boundary

7. Phrase boundary

8. Part of speech disambiguation
sense disambiguation

9. preposition phrase attachment

10. scope: noun, adjectives.

* Ambiguity solver/identifier:

if one WORD is causing confusion → Lexical

can the sentence have diff grammatical structures → Syntactic

is the grammar clear but overall sentence still has multiple meanings?

→ Semantic

Is there issue in he/she/it/they reference? → Anaphoric

Do I need some extra info to understand the sentence?

→ Pragmatic

Empirical Laws

1. Types & Tokens

If in a lang., or the same word occurs multiple times, we call them the same type.

ex: i) She sells sea shells at the sea shore.

Tokens: no. of unique words: 8

Type: ~~no.~~ unique words & which are repeating

She: 1

sells: 1

sea: 2

shells: 1

at: 1

the: 1

shore: 1

Type-token ratio (TTR): type / token

$$TTR(\text{She}) = 1/8$$

$$TTR(\text{sells}) = 1/8$$

$$TTR(\text{sea}) = 2/8$$

$$TTR(\text{shells}) = 1/8$$

$$TTR(\text{at}) = 1/8$$

$$TTR(\text{the}) = 1/8$$

$$TTR(\text{shore}) = 1/8$$

Q. Calculate tokens & type of every token for the given sentence.

i) 'The point that I am going to point out from my point of view is the point that you will definitely point from your point of view if you understand my point.'

Tokens: 31

Type: The: 2

my: 2

if: 1

point: 7

of: 2

understand: 1

that: 2

view: 2

I: 1

is: 1

am: 1

you: 2

going: 1

will: 1

to: 1

definitely: 1

from: 2

your: 1

TTR: ~~??~~

ii) 'kal mein market gaya tha aur market se sabzi khareedi, phir main ghar aaya aur ghar par khana banaya.
Calculate tokens, type (market) & type (main)

Tokens: 19

type (market): 2

type (main): 1

iii) Don't judge the state-of-the-art model by its size, the model's size doesn't define the model.
Calculate tokens & type(model)

(this word count in role type)

Tokens: 218

type(model): 82

TTR(model) = $2/21$

→ Type of Token Ratio (TTR)

ex: 'To be or not to be, that is the question'

Calc. token & TTR *→ how many words are repeating*

: Tokens: 10; count of type = 2; TTR = $2/10$

type(to): 2, type(be), type(the) = 2

TTR is the ratio of no. of different words (types) to the no. of running words (tokens) in a given text of purpose.

$$TTR = \frac{\text{types}}{\text{tokens}}$$

tokens

Q. 'The dog ran to the park. The dog saw a cat. The cat ran away & the dog barked.'

Given the word dog, calc the relative freq. of dog & coming to the sentence given.

$$\rightarrow P(\text{dog}) = \frac{\text{count}(\text{dog})}{N} \quad \text{total no. of tokens.}$$

tokens = 19

count(dog) = 3

$$P(\text{dog}) = \frac{3}{19}$$

Q. smart Assist ex:

'online shopping has become increasingly popular. Many customers purchase products through mobile applications. Some customers also read product reviews before making a purchase.'

tokens = 37

type = 28

$$TTR = \frac{\text{types}}{\text{tokens}} = \frac{28}{37}$$

No sums at this just know in Heap's

Empirical Laws

I) Zipf's Law.

$$f \propto \frac{1}{n} \quad f \cdot n = K$$

$$m \propto \frac{1}{\sqrt{n}} \quad ; \quad m \propto \sqrt{f} \quad m: \text{no. of meanings.}$$

II) Heap's Law.

$$|V| = K \times N^\beta \quad (\beta = 0.4 - 0.6)$$

EX: $n_1 = 20, f = 1896$

$$n_2 = 8, f_2 = ?$$

$$f = \frac{1}{n}$$

$$f_1 \cdot n_1 = K$$

$$1896 \times 20 = K$$

$$f_2 \cdot n_2 = K$$

$$f_2 \cdot 8 =$$

$$\therefore f_2 = 4740$$

EX: $m_1 = 20; m_2 = 30$

$$n_1 : n_2$$

$$\frac{1}{\sqrt{m_1}} : \frac{1}{\sqrt{m_2}}$$

$$\frac{1}{\sqrt{20}} : \frac{1}{\sqrt{30}}$$

$$\sqrt{30} : \sqrt{20}$$

$$9 : 4$$

$$\frac{m_1}{m_2} = \frac{\sqrt{n_1}}{\sqrt{n_2}}$$

$$\frac{20}{30} = \frac{\sqrt{n_1}}{\sqrt{n_2}}$$

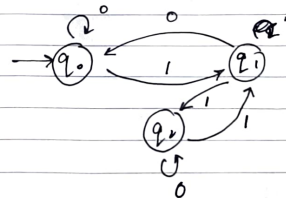
$$\left(\frac{20}{30}\right)^2 = \frac{n_1}{n_2}$$

$$\frac{400}{900} = \frac{n_1}{n_2}$$

$$n_1 : n_2 = 9 : 4$$

EX: DFA.

$$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_1\})$$



→ Why is FST preferable to FSA for morphological analysis

Morphological analysis often requires a mapping b/w
surface word & morphological representation.
cats → cat + plural

FSA: recognizes whether word belongs to a lang.

FST: represents the relationship/mapping b/w
the surface form & its morphological representation

* Porter Stemmer Algo.

ex: gradually
c v e e c c e e v
c v c v c v
c v c v c v

$[c] [vc]^m [v]$

* when y is after
a consonant, then
it is considered
as a vowel

$([c] \text{ or } [vc]) ([vc]^m) [v] \text{ or } [c]$

ex: youth fullnesses
c v v e c c v e c c v c c v c

c v c v c v c v c
c $[vc]^4$

$\therefore m = 4$

ex: pseudopseudohypoparathyroidism
c c v v c v c c v v c v c v c v c c v e r v e v c c

c v c v c v c v c v c v c v c v c v c
c $[vc]^{11}$

$\therefore m = 11$

* Stemming:

Steps:

1A:

Rules

sses → ss

ies → i

ss → ss

s → 's

Examples:

case ss es →

1B

Rules

$(m > 0) EE \rightarrow EE$

$(*V*)ED \rightarrow ' '$

$(*V*)ING \rightarrow ''$

feed $\rightarrow$ feed

agreed $\rightarrow$ agree

plastered $\rightarrow$ plaster

bled $\rightarrow$ bleed

monitoring $\rightarrow$ motor

bled $\rightarrow$ bleed

sing $\rightarrow$ s

1C

Rules

$(*V*)Y \rightarrow I$

Ex:

happy $\rightarrow$ happy

sky $\rightarrow$ sky

* Snowball Algorithm

- unigram model

$$P(W_1 W_2 \dots W_n) = \prod_1 P(W_i)$$

- bigram model

$$P(W_i | W_1 W_2 \dots W_{i-1}) = P(W_i | W_{i-1})$$

EX: Calc. the unigram prob. of 'do.'

$$P(\text{do}) = \frac{2}{60} = \frac{1}{30}$$

$P(w_i) = \frac{\text{count of } w_i}{\text{total words}}$

$$P(W_i | W_{i-1}) = \frac{\text{count}(W_{i-1}, W_i)}{\text{count}(W_{i-1})}$$

$$P(I / \langle s \rangle) = \frac{\text{count}(\langle s \rangle, I)}{\text{count}(\langle s \rangle)}$$

monitoring rest of statement

EX: $P(I | \langle s \rangle) = 2/3$

$$P(\langle s \rangle | \text{here}) = 1/3$$

$$P(\text{would} | \text{I}) = 1/3$$

I am here
who am I
I would like to know

EX.

Consider $S \rightarrow w_1 w_2 w_3 w_4$
 'I live in Mumbai and I love + _____'

P(W)

$$\alpha = P(w_1, w_2, w_3, \dots, w_n) \quad (1)$$

P(S) joint prob./ sentence prob

P(W₄) $\rightarrow$ conditional prob/prob of upcoming word

Apply chain rule.

$$P(B/A) = \frac{P(A, B)}{P(A)}$$

apply chain rule on eqⁿ (2)

$$P(x_1, x_2, \dots, x_n) = P(x_1) P(x_2 | x_1) P(x_3 | x_1, x_2) \dots P(x_n | x_1, \dots, x_{n-1})$$

$$\approx P(w_1, w_2, \dots, w_n) = \prod_i P(w_i | w_1, w_2, \dots, w_{n-1}) \quad (3)$$

Apply Markov assumption on eqⁿ (3)

says that

{ Markov Assumption in the prob. of next word
 depends only on a limited no. of recent
 previous words, not on entire history. }

for that we use N-GRAM language model.

i) unigram model

$$P(w_1, w_2, \dots, w_n) \approx \prod_i P(w_i)$$

$$P(w_i) = \frac{\text{count}(w_i)}{N(\text{tokens})}$$

ONLY IN UNIGRAM, WE WON'T CONSIDER THE SENTENCE BOUNDARY MARKERS. <S> / <1>

i)

2. Calc. unigram prob.
 of given words.

EX:

<S> I like NLP </S>

<S> I like Blockchain </S>

<S> Sam likes NLP </S>

 $\rightarrow$

Step 1:	Word	Frequency
	I	2
	like	2
	NLP	2
	Blockchain	1
	Sam	1
	likes	1
Sentence boundary markers	<S>	3
	</S>	3

tokens = 159

} not used here.

Step 2: Calc. Prob.

$$P(w_i) = \frac{c(w_i)}{N}$$

$$P(I) = \frac{2}{159} \quad \frac{2}{9}$$

$$P(\text{like}) = \frac{2}{159} \quad \frac{2}{9}$$

$$P(\text{NLP}) = \frac{2}{159} \quad \frac{2}{9}$$

$$P(\text{Blockchain}) = \frac{1}{159} \quad \frac{1}{9}$$

$$P(\text{Sam}) = \frac{1}{159} \quad \frac{1}{9}$$

$$P(\text{likes}) = \frac{1}{159} \quad \frac{1}{9}$$

P

ii)

Calc. sentence prob. of 'I like NLP'.

w₁ w₂ w₃

$$P(S) = \prod_i P(w_i) = P(w_1) P(w_2) P(w_3) \\ = \frac{2}{9} \times \frac{2}{9} \times \frac{2}{9}$$

Ex: Calc. sentence prob. of 'I like NLP'.

ii) Bigram model

considers n-1 word history.

$$P(w_i | w_1, w_2, \dots, w_{n-1}) \approx P(w_i | w_{i-1})$$

$$= \frac{\text{count}(w_{i-1}, w_i)}{c(w_{i-1})}$$

Ex. Calc. bigram prob. $\langle s \rangle$ I like NLP $\langle s \rangle$.

w1 $P(I / \langle s \rangle) = \frac{c(\langle s \rangle, I)}{c(\langle s \rangle)}$

$$= \frac{2}{3}$$

w2 $P(\text{like} / I) = \frac{c(I, \text{like})}{c(I)}$

$$= \frac{2}{2} = 1$$

w3 $P(\text{NLP} / \text{like}) = \frac{c(\text{like}, \text{NLP})}{c(\text{like})}$

$$= \frac{1}{2}$$

w4 $P(\langle s \rangle / \text{NLP}) = \frac{c(\text{NLP}, \langle s \rangle)}{c(\text{NLP})}$

$$= \frac{2}{2} = 1$$

$$P(\text{sentence}) = \cancel{c(\langle s \rangle)} P(w_1) P(w_2) P(w_3) P(w_4)$$

$$= \frac{2}{3} \times 1 \times \frac{1}{2} \times 1$$

$$= \frac{1}{3} \approx 0.33$$

iii) Trigram model

↳ two-words history

$$P(w_i, w_{i+1}, w_{i+2}) = \prod_{i=1}^n P(w_i | w_1, w_2)$$

$$= \frac{c(w_{i-2}, w_{i-1}, w_i)}{c(w_{i-2}, w_{i-1})}$$

iv) Four gram model

↳ three-words history

$$P(w_1, w_2, w_3, w_4) = \prod_{i=1}^n P(w_i | w_1, w_2, w_3)$$

Ex. What is the most probable next word predicted in the model from the fol. word sequences for the given purpose:

$\langle s \rangle$ I am Henry $\langle /s \rangle$

$\langle s \rangle$ I like college $\langle /s \rangle$

$\langle s \rangle$ Do Henry like college $\langle /s \rangle$

$\langle s \rangle$ Henry I am $\langle /s \rangle$

$\langle s \rangle$ Do I like Henry $\langle /s \rangle$

$\langle s \rangle$ Do I like college $\langle /s \rangle$

$\langle s \rangle$ I do like college $\langle /s \rangle$

⇒ i) Do?

ii) $\langle s \rangle$ I like Henry?

iii) $\langle s \rangle$ Do I like?

iv) $\langle s \rangle$ Do I like college?

bigram

bigram

trigram

four gram

no numericals on tri & four gram.

Ans: 3) freq. table

word	freq.
I	6
am	2
Henry	5
like	5
college	3
Do	4
<S>	7
</S>	7

tokens = 39

1) Calc. Prob. acc. to questions.

i) Do ?

$$P(W_i | W_{i-1}) = \frac{c(W_{i-1}, W_i)}{c(W_{i-1})}$$

$$P(\text{Henry}/\text{Do}) = \frac{c(\text{Do}, \text{Henry})}{c(\text{Do})} = \frac{1}{4} \approx 0.25$$

$$P(I/\text{Do}) = \frac{c(\text{Do}, I)}{c(\text{Do})} = \frac{2}{4} = 0.5$$

$$P(\text{Like}/\text{Do}) = \frac{c(\text{Do}, \text{Like})}{c(\text{Do})} = \frac{1}{4} \approx 0.25$$

$\therefore$ 'I' is the most probable word to come after 'Do'.

ii) <S> I like Henry ?

$$P(W_i | W_{i-1}) = \frac{c(W_{i-1}, W_i)}{c(W_{i-1})}$$

$$P(<S>/\text{Henry}) = \frac{c(\text{Henry}, <S>)}{c(\text{Henry})} = \frac{3}{5}$$

$$P(\text{like}/\text{Henry}) = \frac{c(\text{Henry}, \text{like})}{c(\text{Henry})} = \frac{1}{5}$$

$$P(I/\text{Henry}) = \frac{c(\text{Henry}, I)}{c(\text{Henry})} = \frac{1}{5}$$

$\therefore$ </S> is the most probable word.

iii) <S> Do I like ?

trigram.

$$c(I, \text{like}) = 3 \quad P(W_i | W_{i-1}, W_{i-2}) = \frac{c(W_{i-2}, W_{i-1}, W_i)}{c(W_{i-2}, W_{i-1})}$$

$$P(\text{College} | I, \text{like}) = \frac{c(I, \text{like}, \text{College})}{c(I, \text{like})} = \frac{2}{3}$$

$$\frac{P(\text{Henry} | I, \text{like})}{P(\text{Henry} | \text{like}, I)} = \frac{c(I, \text{like}, \text{Henry})}{c(I, \text{like})} = \frac{1}{3}$$

$\therefore$ 'College' is most probable word.

iv) <S> Do I like college ?

fourgram

$$P(W_i | W_{i-1}, W_{i-2}, W_{i-3}) = \frac{c(W_{i-3}, W_{i-2}, W_{i-1}, W_i)}{c(W_{i-3}, W_{i-2}, W_{i-1})}$$

$$P(</S> | \text{Do}, \text{like}, \text{college}, I, \text{Do}) = \frac{c(\text{Do}, \text{like}, \text{college}, </S>)}{c(\text{Do}, \text{like}, \text{college})} = 1$$

EX: Which of the fol. sentences get higher prob. using Bigram

<S> I like college </S>

<S> Do I like Henry </S>

$\Rightarrow$

word	freq.
<S>	2
</S>	2
I	2
like	2
college	1
Do	1
Henry	1

tokens = 11

$$(i) P(S) = P(I / <S>) * P(Like / I) * P(College / Like) * P(<S> / College)$$

$$= P(W_i / W_{i-1}) = \frac{c(W_{i-1}, W_i)}{c(W_{i-1})}$$

$$P(S) = \frac{c(<S>, I)}{c(<S>)} * \dots$$

$$= \frac{3}{7} * \frac{3}{6} * \frac{3}{5} * \frac{3}{3}$$

$$= \frac{3}{7} * \frac{1}{2} * \frac{3}{5}$$

$$P(S) = 0.13$$

$$(ii) P(S_2) = P(DO / <S>) * P(I / DO) * P(Like / I) + P(Henry / Like) * P(<S> / Henry)$$

$$P(W_i / W_{i-1}) = \frac{c(W_{i-1}, W_i)}{c(W_{i-1})}$$

$$P(S_2) =$$

$$P(S_2) = \frac{3}{7} * \frac{2}{4} * \frac{3}{6} * \frac{2}{5} * \frac{3}{5}$$

$$P(S_2) = 0.0257$$

First statement is having highest prob.

Zero probability Problem

If a sentence contains a zero-prob word / n-gram, its entire sentence probability can become zero.

Smoothing is required when we need to prevent zero-probabilities for unseen words / n-grams.

→ Solⁿ : Smoothing techniques

i) Laplace Smoothing / Add-one smoothing

it adds 1 to every n-gram count to avoid zero prob.

when we want to segregate words

→ unigram:

$$P(W | \text{class}) = \frac{c(W) + 1}{N + |V|}$$

used for

calc prob

of sentences

$$\text{bigram: } P(W_i | W_{i-1}) = \frac{c(W_{i-1}, W_i) + 1}{c(W_{i-1}) + |V|}$$

c → count ; |V| → vocabulary size / type

N → token

EX: For given corpus compute smooth prob. of the test sentence

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like Sam </s>

Test sentence:

<s> I am Sam </s>

Word	Freq.	$p(w_i w_{i-1}) = \frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + V}$
<s>	3	
</s>	3	
I	3	$V = \text{type} = 6$
am	2	
Sam	3	

$P(\text{test} | \text{statement})$

$$P(I / <s>) = \frac{c(<s>, I) + 1}{c(<s>) + V} = \frac{2 + 1}{3 + 6} = \frac{3}{9} = \frac{1}{3}$$

$$P(\text{am} / I) = \frac{c(I, \text{am}) + 1}{c(I) + V} = \frac{2 + 1}{3 + 6} = \frac{1}{3}$$

$$P(\text{Sam} / \text{am}) = \frac{c(\text{am}, \text{Sam}) + 1}{c(\text{am}) + V} = \frac{2 + 1}{5 + 4} = \frac{1}{3}$$

$$P(</s> / \text{Sam}) = \frac{c(\text{Sam}, </s>) + 1}{c(\text{Sam}) + V} = \frac{2 + 1}{3 + 6} = \frac{1}{3}$$

$$P(S) = \frac{1}{3} \times \frac{1}{3} \times \frac{1}{4} \times \frac{1}{3} = 0.009$$

EX: For given corpus:

<s> I am here </s>

<s> I like it </s>

<s> I am happy </s>

compute smooth prob. for:

i) $P(\text{here} | I)$

ii) $P(\text{it} | I)$

iii) $P(\text{happy} | I)$

iv) $P(I | I)$

Word	Freq.	$V = 6$
<s>	3	$N = 9$
</s>	3	
I	3	
am	2	
here	1	
like	1	
it	1	
happy	1	

$$P(W | \text{class}) = \frac{c(W) + 1}{N + V}$$

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + V}$$

$$P(\text{here} | I) = \frac{1 + 1}{3 + 6} = \frac{2}{9}$$

$$P(\text{it} | I) = \frac{1 + 1}{6 + 9} = \frac{2}{15}$$

$$P(\text{happy} | I) = \frac{1 + 1}{3 + 6} = \frac{2}{9}$$

$$P(I | I) = \frac{0 + 1}{3 + 6} = \frac{1}{9}$$

$$P(I | I) = \frac{0 + 1}{3 + 6} = \frac{1}{9}$$

Since <class> is not mentioned, we use bigram.

EX: Naive Bayes Spam Filter.
 vocabulary size = 8
 Training statistics: Test if ~~test~~ given email
 is spam/ham.

Word	Count in SPAM
Free	6
Money	5
win	4
click	3
Prize	2
meeting	0
Project	0
Launch	0
Total tokens	20

class: spam
 ham

Test email: 'win free money meeting'.

$$\rightarrow V = 8, N = 20$$

$$P(W | \text{class}) = \frac{c(W) + 1}{N + V}$$

$$P(\text{win} | \text{SPAM}) = \frac{4 + 1}{8 + 20} = \frac{5}{28}$$

$$P(\text{free} | \text{SPAM}) = \frac{6 + 1}{8 + 20} = \frac{7}{28} = \frac{1}{4}$$

$$P(\text{money} | \text{SPAM}) = \frac{5 + 1}{8 + 20} = \frac{6}{28}$$

$$P(\text{meeting} | \text{SPAM}) = \frac{0 + 1}{28} = \frac{1}{28}$$

$$P(\text{email} | \text{SPAM}) = \frac{5}{28} \times \frac{7}{28} \times \frac{6}{28} \times \frac{1}{28} = 3.42 \times 10^{-4}$$

EX: Sentiment Analysis of Product Review.
 Classify a review as POSITIVE or NEGATIVE using
 Naive Bayes. $N = 25, V = 10$

word	^{freq} count in POSITIVE
great	9
love	6
excellent	5
perfect	4
amazing	2
terrible	0
refund	0
broken	0

Test sentence:
 'product broken,
 please refund!'

ii) Good Turing smoothing tech:

1. Good Turing smoothing prob. for an observed in n -gram

$$P_{GT}^* = \frac{C^*}{N}$$

2. Prob. mass for unseen words

$$P_{GT}^*(o) = \frac{N_1}{N} \quad \begin{array}{l} \nearrow \text{freq}^* \text{ of words occur once} \\ \searrow \text{total words} \end{array}$$

3. Adjusted count / good Turing count

$$C^* = (C+1) \frac{N_c+1}{N_c}$$

Ex: Construct

4. Freqⁿ of frequencies (N_c) table

$$N_c = \frac{N_1}{N}$$

EX: Construct frequency of frequencies (N_c) table for $C = 1, 2, 3, 4$. Calculate total prob. mass assigned to unseen words.

Calc. good Turing smooth prob. of word 'cat'.

⇒ unigram:

Training corpus.

Word	count
the	4
cat	3
dog	3
runs	2
sits	2
red	1
blue	1
big	1
small	1
house	1
car	1

$N = 20$

a) freqⁿ of frequencies table (N_c)

Occurance	words	N_c
1 time	red, blue, big, small, house, car	6
2 times	runs, sits	2
3 times	cat, dog	2
4 times	the	1

So, $N_1 = 6$, $N_2 = 2$, $N_3 = 2$, $N_4 = 1$

b) Prob. of unseen word

$$P_{GT}^* = \frac{C^*}{N} = \frac{N_1}{N} \quad \begin{array}{l} C^* \text{ is the count of words} \\ \text{which occurs one time /} \\ \text{the least count of time} \end{array}$$

$$= \frac{6}{20}$$

$$P_{GT}^* = 0.30$$

30% of prob. mass is assigned to unseen words.

c)

~~Recall~~ GT prob. for 'cat'

$$\text{count} = 3$$

good turing adjusted (discount) count

$$c^* = (c+1) \frac{N_c+1}{N_c}$$

$$= (3+1) \frac{N_4}{N_3}$$

$$= 4 \left(\frac{1}{2} \right)$$

$$c^* = 2$$

$$P(\text{cat}) = \frac{c^*}{N}$$

$$= \frac{2}{20}$$

$$P(\text{cat}) = 0.1$$

EX: Construct freq. of freq. table. Calc. total prob. mass assigned to ^{un}seen words. bigrams. Calc. good turing smoothing prob. of 'I want'?

Training Corpus:

Word	Count
I like	4
I love	3
I want	2
I need	2
I play	1
I watch	1
I read	1
I study	1

$$\Rightarrow N = 15$$

a) Nc table

occurrence	bigrams	Nc.
1 time	I play, I watch, I read, I study	4
2 times	I want, I need	2
3 times	I love	3
4 times	I like	4

$$\text{so, } N_1 = 4, N_2 = 2, N_3 = 3, N_4 = 4$$

b) GT prob. of unseen words.

$$P_{GT}^*(0) = \frac{N_1}{N} = \frac{4}{15} = 0.267$$

Approx 26.7%. prob. mass is reserved for unseen bigrams

c) GT prob for 'I want'
count = 2

$$c^* = (c+1) \frac{N_{c+1}}{N_c}$$

$$= 3 \left(\frac{N_3}{N_2} \right)$$

$$= 4 \times 3 \left(\frac{1}{2} \right)$$

$$c^* = 1.5$$

$$P(\text{want} | I) = \frac{P(I, \text{want})}{P(I)} \quad \text{or} \quad \frac{c^*}{N}$$

$$= \frac{2}{15} \quad = \frac{1.5}{15}$$

$$= 0.133 \quad = 0.1$$

Evaluating Language Model

i) Extrinsic

ii) Intrinsic evaluation:

Perplexity

lower perplexity

↳ better model

perplexity:

for a test set $W = w_1 w_2 \dots w_N$

$$PP(W) = P(w_1, w_2 \dots w_N)^{-1/N}$$

$$= \sqrt[N]{\frac{1}{P(w_1, w_2 \dots w_N)}}$$

; N is given no. of sentence words test in sentence

Ex: Compile ~~pre~~ PP of given corpus using bigram model of the sentence "She likes mangoes"

corpus:

She likes apples. She likes bananas. He likes mangoes.

They eat fruits. She likes grapes.

A: Step 1: write corpus using sentence boundaries

<S> ^ She likes apples </S>

<S> She likes banana </S>

<S> He likes mangoes </S>

<S> They eat fruits </S>

<S> She likes grapes </S>

Step 2: List bigrams in given ^{test} sentences.

[<s>, she], [she, likes], [likes, mangoes],

[mangoes, <s>]

Step 3: Compute bigram prob.

$P(<s>, she)$

$$P(she / <s>) = \frac{c(<s>, she)}{c(<s>)} = \frac{3}{5}$$

$$P(likes / she) = \frac{c(she, likes)}{c(she)} = \frac{3}{3} = 1$$

$$P(mangoes, likes) = \frac{c(likes, mangoes)}{c(likes)} = \frac{1}{4}$$

$$P(<s>, mangoes) = \frac{c(mangoes, <s>)}{c(mangoes)} = \frac{1}{1} = 1$$

$$P(W) = \frac{3}{5} \times 1 \times \frac{1}{4} \times 1 = \frac{3}{20}$$

$\therefore P_{B2}$

Step 4: Compute Perplexity

$$PP(W) = N \sqrt{\frac{1}{P(W)}} \quad \begin{array}{l} N=3 \\ \text{no. of words in} \\ \text{test sentence} \end{array}$$

$$= 3 \sqrt{\frac{1}{3/20}} = 3 \sqrt{\frac{20}{3}}$$

$$PP(W) = 1.8821$$

EX: 2 unigram models.

'The patient feels very weak.'

$$P_A(W) = 0.00008 \quad \text{and} \quad P_B(W) = 0.00002$$

$N=5$

~~Perplexity~~

a) ^{calc.} Perplexity of Model A

$$PP_A(W) = N \sqrt{\frac{1}{P_A(W)}} = 5 \sqrt{\frac{1}{0.00008}} = 6.5975$$

unigram is not good enough, since it takes no word history. so, bigram would be better for these types of questions.

Statistical text representation techniques

→ Bag of words (BoW)
represents sentences in diff forms of vectors.

EX: create vocabulary

D₁: I love NLP

D₂: I love machine learning.

A: Step 1: create vocabulary

[I, love, NLP, machine learning]

Step 2: Count of documents

DOC	I	love	NLP	machine	learning
D ₁	1	1	1	0	0
D ₂	1	1	0	1	1

$$D_1 = [1, 1, 1, 0, 0]$$

$$D_2 = [1, 1, 0, 1, 1]$$

BoW →

⇒ TF-IDF: Term-freq. —

assigns numerical weight to a word based on how frequently the word occurs in a particular doc.

$$TF = \frac{\text{no. of occ. of word in doc}}{\text{total no. of words in doc}}$$

$$IDF = \log \left(\frac{\text{total no. of sentences}}{\text{no. of sentences containing word}} \right)$$

$$TF-IDF = TF \times IDF$$

EX: Compute TF-IDF for given corpus.

D₁: I love NLP

D₂: I love machine learning

D₃: NLP is useful.

A: TF:

D₁: I love NLP

D₂: I love ML

D₃: NLP is useful

word	TF
I	1/3
love	1/3
NLP	1/3

word	TF
I	1/4
love	1/4
machine	1/4
learning	1/4

word	TF
NLP	1/3
is	1/3
useful	1/3

II) IDF:

word	count	IDF
I	2	$\log(3/2)$
love	2	$\log(3/2)$
NLP	2	$\log(3/2)$
machine	1	$\log(3/1)$
learning	1	$\log(3/1)$
is	1	$\log(3/1)$
useful	1	$\log(3/1)$

$$\left. \begin{matrix} \log(3/2) \\ \log(3/2) \\ \log(3/2) \end{matrix} \right\} = 0.1761$$

$$\left. \begin{matrix} \log(3/1) \\ \log(3/1) \\ \log(3/1) \end{matrix} \right\} = 0.4771$$

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

D_1 :

$$I \rightarrow \frac{1}{3} \times 0.1761 = 0.0587$$

$$\text{love} \rightarrow \frac{1}{3} \times 0.1761 = 0.0587$$

$$\text{NLP} \rightarrow \frac{1}{3} \times 0.1761 = 0.0587$$

So,

$$\text{TF-IDF}_{D_1} = [0.0587, 0.0587, 0.0587]$$

D_2 :

$$I \rightarrow \frac{1}{4} \times 0.1761 = 0.044$$

$$\text{love} \rightarrow \frac{1}{4} \times 0.1761 = 0.044$$

$$\text{m/c} \rightarrow \frac{1}{4} \times 0.4771 = 0.1193$$

$$\text{tr} \rightarrow \frac{1}{4} \times 0.4771 = 0.1193$$

So,

$$\text{TF-IDF}_{D_2} = [0.044, 0.044, 0.1193, 0.1193]$$

D_3 :

$$\text{NLP} \rightarrow \frac{1}{3} \times 0.1761 = 0.0587$$

$$W \rightarrow \frac{1}{3} \times 0.4771 = 0.1590$$

$$\text{useful} \rightarrow \frac{1}{3} \times 0.4771 = 0.1590$$

So,

$$\text{TF-IDF}_{D_3} = [0.0587, 0.1590, 0.1590]$$

MODULE 3: SYNTAX ANALYSIS & SEQUENCE MODELLING

* Syntax Analysis 3rd phase of NLP.

→ POS tagging (Part of speech)

It is a process that attaches each word in sentence with a word

* HMM for POS tagging (Hidden Markov Model)

$W = w_1, w_2, w_3, \dots, w_N \rightarrow$ words in the corpus (observed)

$T = t_1, t_2, t_3, \dots, t_N \rightarrow$ the corresponding tags (unknown)

ex. I love pizza

w_1	w_2	w_3
I	love	pizza
$\downarrow$	$\downarrow$	$\downarrow$
t_1	t_2	t_3
PR	VB	NN

To find unambiguous tag sequence, given word sequence we have to find $\hat{T}$ or T^* which is the most probable tag sequence. It is found from argmax or probability of T given W i.e. that is among all the possible sequences of POS tags that the word sequence can take the max prob.

$$\hat{T} = \underset{T}{\operatorname{argmax}} P(T/W) \quad \text{--- (i)}$$

Apply Bayes Theorem of conditional prob. or eqⁿ (i)

$$P(x/y) = \frac{P(y/x) P(x)}{P(y)}$$

$$\hat{T} = \underset{T}{\operatorname{argmax}} \frac{P(W(T) P(T))}{P(W)}$$

which can be rewritten as (for n words)

$$\hat{T} = \underset{T}{\operatorname{argmax}} \frac{P(W_i/t_i) P(t_i)}{P(W_i)} \quad \text{--- (ii)}$$

Simplify eqⁿ (ii) by dropping the denominator ' $P(W_i)$ ' (because set of words will be similar for all)

$$\hat{T} = \underset{T}{\operatorname{argmax}} \underbrace{P(W_i/t_i)}_{\substack{\text{likelihood} \\ \text{prob of word} \\ \text{sequence for given} \\ \text{tag sequence}}} \underbrace{P(t_i)}_{\substack{\text{prior prob.} \\ \text{of tag sequence}}} \quad \text{--- (iii)}$$

Apply chainrule on prob. $P(t_i)$

$$P(t_i) = P(t_1) P(t_2/t_1) P(t_3/t_1, t_2) \dots P(t_n/t_1, t_2, \dots, t_{n-1}) \quad \text{--- (iv)}$$

Apply Markov's assumption on eqⁿ (iv)

$$P(t_i) = P(t_1) P(t_2/t_1) P(t_3/t_2) \dots P(t_n/t_{n-1})$$

$$= \prod_{i=1}^n P(t_i/t_{i-1}) \quad \text{--- (v)} \quad \left\{ \begin{array}{l} \text{Bigram} \\ \text{Assumption} \end{array} \right\}$$

Apply chain rule on likelihood prob. ' $P(W_i/t_i)$ ' from eqⁿ (iii)

$$P(W_i/t_i) = P(w_1, w_2, \dots, w_n | t_1, t_2, \dots, t_n)$$

$$= P(w_1 | t_1, t_2, \dots, t_n) P(w_2 | w_1, t_1, t_2, \dots, t_n) \dots P(w_n | w_1, \dots, w_{n-1}, t_1, \dots, t_n) \quad \text{--- (vi)}$$

Apply Markov's assumption on eqⁿ (vi)

$$P(W_i/t_i) = \prod_{i=1}^n P(w_i | t_i) \quad \text{--- (vii)} \quad \left\{ \begin{array}{l} \text{Lexical Prob. assumption} \end{array} \right\}$$

Substitute values of $eq^n(V)$ & (W) in $eq^n(N)$

$$\hat{t}_i = \underset{t_i}{\operatorname{argmax}} \underbrace{\prod_{i=1}^n P(W_i | t_i)}_{\text{Emission prob.}} \underbrace{P(t_i | t_{i-1})}_{\text{State transition prob.}}$$

$$P(W_i | t_i) = \frac{c(t_i, W_i)}{c(t_i)}$$

$$P(t_i | t_{i-1}) = \frac{c(t_{i-1}, t_i)}{c(t_{i-1})}$$

Ex: Calc. emission & transition prob. for given corpus.
 $\langle s \rangle$ Book/V a/DET car/N $\langle /s \rangle$
 $\langle s \rangle$ Park/V ~~the~~ the/DET car/N $\langle /s \rangle$
 $\langle s \rangle$ The/DET book/N in/V in/Prep the/DET car/N $\langle /s \rangle$
 $\langle s \rangle$ THE/DET car/N is/V in/Prep a/DET park/N $\langle /s \rangle$
 Bigram HMM

Ans. Step 1: calc. word-freq table

word	freq.
Book	2
a	2
car	4
Park	2
the	4
in	2

in	2
P	

Step 2: create ~~initial~~ prob matrix

	(4) V	(6) Det	(6) N	Prp(2)
Book	1/4	0	1/6	0
a	0	2/6	0	0
car	0	0	4/6	0
park	1/4	0	1/6	0
in	2/4	0	0	0
in	0	0	0	2/2 = 1
the	0	4/6	0	0

HMM problem: 1st initial emission prob.
2nd state transition prob.
3rd initial pr prob. matrix (IPM)

Step 3: create transition prob. matrix

	N	V	DET	Prp	<S>
<S> (4)	0	2/4	2/4	0	0
N (6)	0	2/6	0	0	4/6
V (4)	0	0	2/4	2/4	0
Det (6)	4/6	0	0	0	0
Prp (2)	0	0	2/2=1	0	0

Ex. Make an HMM table for wrt given matrices.

i/p:

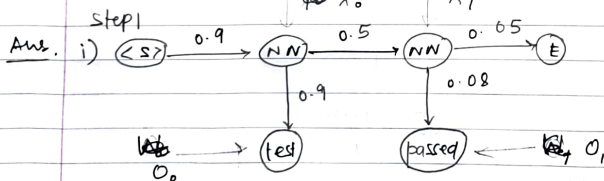
Test passed

o/p:

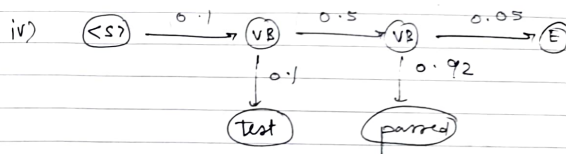
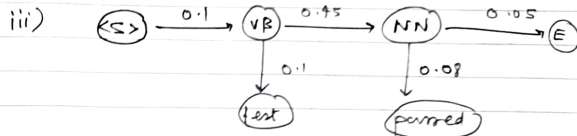
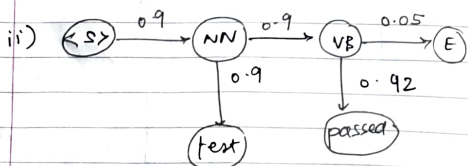
['Test', 'NN'], ['passed', 'VB']

0 in slides

Generate HMM table for given sentence
(Test passed)



IPM = $\pi = \begin{bmatrix} 0.9 & 0.1 \end{bmatrix}$



Step 2: HMM table

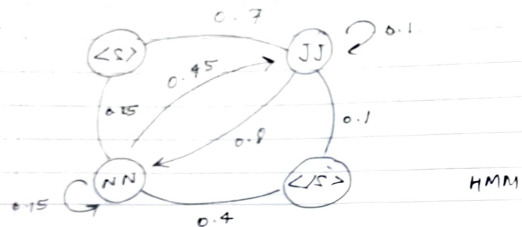
~~is~~ ~~VB~~ ~~VB~~ ~~VB~~

X_0	X_1	$\pi(X_0)$	$P(O_0, X_0)$	$P(X_0, X_1)$	$P(O_1, X_1)$	$P(X_1)$	P_2	Normalized
NN	NN							
NN	VB							
VB	NN							
VB	VB							

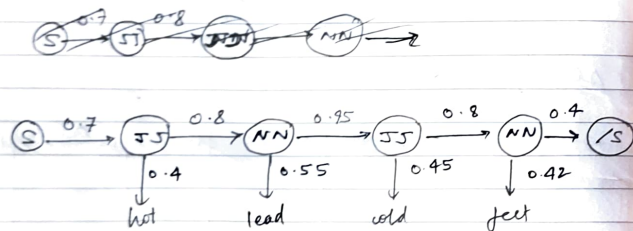
X_0	X_1	$\pi(X_0)$	$P(O_0, X_0)$	$P(X_0, X_1)$	$P(O_1, X_1)$	$P(X_1)$	P_2	Normalized
NN	NN	0.9	0.9	0.05	0.08	0.05	0.00162	0.00475
NN	VB	0.9	0.9	0.90	0.92	0.05	0.033534	0.98313
VB	NN	0.1	0.1	0.45	0.08	0.05	0.00018	0.00528
VB	VB	0.1	0.1	0.5	0.92	0.05	0.000230	0.00674

$\frac{Pr}{\sum Pr}$

Ex.



⇒



$$P() = 0.7 \times 0.8 \times 0.45 \times 0.8 \times 0.4 \times 0.4 \times 0.55 \times 0.45 \times 0.42$$

* Viterbi Algorithm

O = seq. of observations

 $O_1, O_2, O_3, \dots, O_T$ = individual observations

T = no. of observations.

$$H = \lambda$$

input:

- observations of T length
- state graph

Q

for given training corpus:

<S> Martin Justin can watch Will. <E>

<S> Spot will watch Martin <E>

<S> will ~~spot~~ Justin spot Martin <E>

<S> Martin will Pat Spot <E>

Calc. most probable tag sequence using Viterbi

algo. & calc path prob. for given test sentence

'Justin will spot will'

Ans step 1: Assign correct pos tags to training corpus

N N M V N

N M V N

M N V N

N M V N

where, assume : N $\rightarrow$ Noun

M $\rightarrow$ modal verb

V $\rightarrow$ verb.

Step 2: Calc. emission / observation prob. matrix

	N(4)	M(4)	V(4)
Martin	4/9	0	0
Justin	2/9	0	0
Will	1/9	3/4	0
Spot	2/9	0	1/4
can	0	1/4	0
watch	0	0	2/4
pat	0	0	1/4

step 3: Calc. state transition prob. matrix

	N	M	V	<E>
<S>	3/4	1/4	0	0
N	1/9	1/9	1/9	4/9
M	1/4	0	3/4	0
V	1	0	0	0

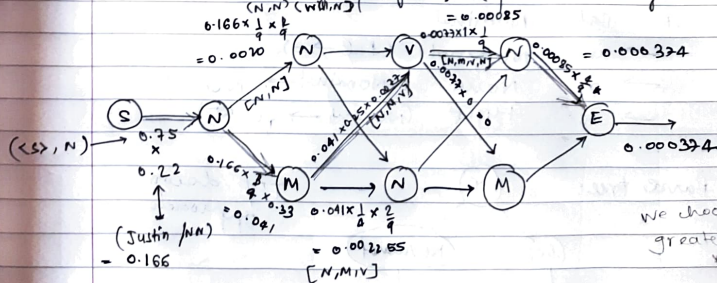
Step 4: State graphs.

<S> Justin will spot Will <E>

N N V N M M

 $1 \times 2 \times 2 \times 2 = 8 \text{ ways}$

Draw state graph for all possible tags.



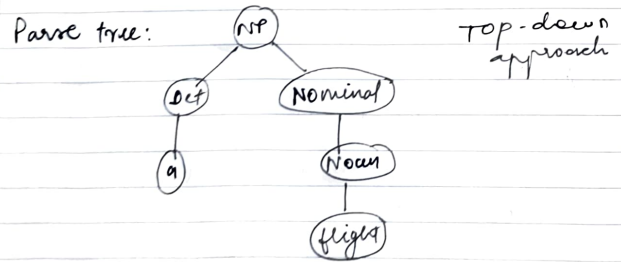
we choose the greater value path.

<S> Justin will spot Will <E>
[N] [M] [V] [N]

- Parsing
a method of analyzing

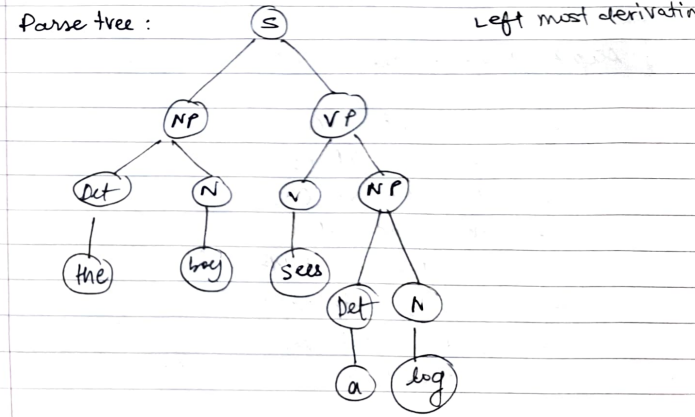
Ex: Input : a flight
NP $\rightarrow$ Det Nominal
Nominal $\rightarrow$ Noun / Nominal Noun
Det $\rightarrow$ a / an / the
Noun $\rightarrow$ cat / ground / butterfly

Ans NP $\rightarrow$ Det Nominal
 $\rightarrow$ a Nominal (Det $\rightarrow$ a)
 $\rightarrow$ a Noun (Nominal $\rightarrow$ Noun)
 $\rightarrow$ a Flight (Noun $\rightarrow$ flight)



Ex: I/p : The boy sees a dog.
S $\rightarrow$ NP VP
NP $\rightarrow$ Det N
VP $\rightarrow$ V NP
Det $\rightarrow$ the/a
N $\rightarrow$ boy/dog
V $\rightarrow$ sees/likes

Ans: S $\rightarrow$ NP VP
 $\rightarrow$ Det N VP (NP $\rightarrow$ Det N)
 $\rightarrow$ the N VP (Det $\rightarrow$ the)
 $\rightarrow$ the boy VP (N $\rightarrow$ boy)
 $\rightarrow$ the boy V NP (VP $\rightarrow$ V NP)
 $\rightarrow$ the boy sees NP (V $\rightarrow$ sees)
 $\rightarrow$ the boy sees Det N (NP $\rightarrow$ Det N)
 $\rightarrow$ the boy sees a N (Det $\rightarrow$ a)
 $\rightarrow$ the boy sees a dog (N $\rightarrow$ dog)



Ex: Perform left-most derivation & compute parse tree to check the string:

ip: "The Panda eats leaves"

$S \rightarrow NP VP$

$NP \rightarrow DT NN$

$VP \rightarrow VB NN$

$DT \rightarrow a/the$

$NN \rightarrow man/woman/Panda/lion/leaves.$

$VB \rightarrow eats/shoots/leaves.$

Ans

$S \rightarrow NP VP$

Step 1: Label the grammar rules

$S \rightarrow NP VP^1$

$NP \rightarrow DT^1 NN^2$

$VP \rightarrow VB^3 NN^3$

$DT \rightarrow a^4/the^5$

$NN \rightarrow man^6/woman^7/Panda^8/lion^9/leaves^{10}$

$VB \rightarrow eats^{11}/shoots^{12}/leaves^{13}$

Step 2: Perform left-most derivation

$S \rightarrow NP VP \quad (1)$

$\rightarrow DT NN VP \quad (2)$

$\rightarrow The NN VP \quad (5)$

$\rightarrow The Panda VP \quad (8)$

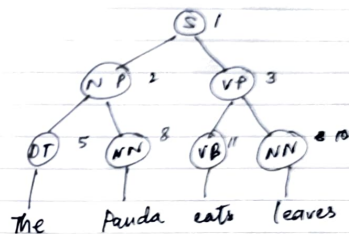
$\rightarrow The Panda VB NN \quad (3)$

$\rightarrow The Panda eats NN \quad (11)$

$\rightarrow The Panda eats NN \quad (3)$

$\rightarrow The Panda eats leaves \quad (10)$

Step 3: Generate parse tree



EX:

— " —

ip: The teacher gives a book to the student

$S \rightarrow NP VP^1$

$NP \rightarrow DT NN^2$

$VP \rightarrow VB NP PP^3$

$PP \rightarrow Prep NP^4$

$DT \rightarrow The^5/a^6$

$NN \rightarrow teacher^7/student^8/book^9$

$VB \rightarrow gives^{10}$

$Prep \rightarrow to^{11}$

Ans

Step 1: Label

Step 2: Perform Left-most

$S \rightarrow NP VP \quad (1)$

$\rightarrow DT NN VP \quad (2)$

$\rightarrow The NN VP \quad (5)$

$\rightarrow The teacher VP \quad (7)$

$\rightarrow The teacher VB NP PP \quad (3)$

$\rightarrow The teacher gives NP PP \quad (10)$

$\rightarrow The teacher gives DT NN PP \quad (2)$

$a NN PP \quad (9)$

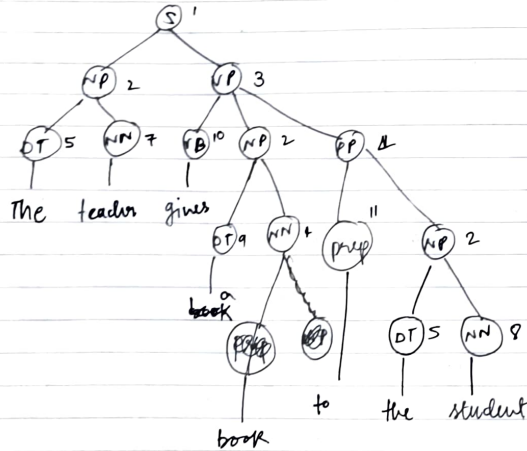
$a book Prep NP PP \quad (4)$

$a look to NP PP \quad (11)$

$a look to DT NN \quad (13)$

→ the teacher gives a book to the student (S)
 →

Step 3: parse tree



(*)

Chomsky Normal Form (CNF):

Ex:

$S \rightarrow aSb \mid ab$ ✓

$X \rightarrow YZA$ ✗

$A \rightarrow a$ ✓

$B \rightarrow b$ ✓

- Cocke-Younger-Kasami (CYK or CKY Parser)
 Reach till root of parse tree

(*)

Ex:

Consider the grammar, construct CKY table & parse tree per given i/p sentence:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

$V \rightarrow \text{includes}$

$Det \rightarrow \text{The}$

$Det \rightarrow a$

$N \rightarrow \text{meal}$

$N \rightarrow \text{flight}$

"The flight includes a meal"

Ans.

The	flight	includes	a	meal
Det	NP			S
01	02	03	04	05
	N			
11	12	13	14	15
		V		VP
21	22	23	24	25
		Det		NP
31	32	33	34	35
			N	
41	42	43	44	45

Parse Tree



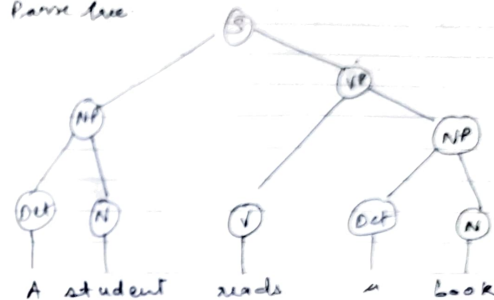
EX: "A student reads a book"

$S \rightarrow NP VP$
 $NP \rightarrow Det N$
 $VP \rightarrow V NP$
 $Det \rightarrow a$
 $N \rightarrow \text{student/book}$
 $V \rightarrow \text{reads}$

Ans

	A	student	reads	a	book
Det	→ NP				→ S
	↑	↑	↑	↑	↑
	N				
			V		VP
				Det	→ NP
					↑
					N

Parse Tree



• (2m)

PCFG: Probabilistic Context Free Grammar.

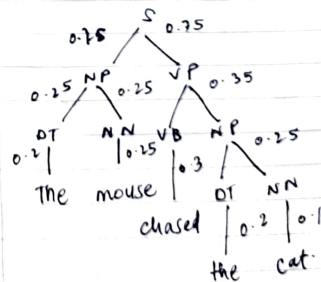
EX:

Compute the prob of parse tree of i/p string 'The mouse chased the cat.'

Given the fol partial PCFG.

$S \rightarrow NP VP \quad 0.75$
 $NP \rightarrow DT NN \quad 0.25$
 $VP \rightarrow VB NP \quad 0.35$
 $NN \rightarrow \text{mouse } 0.25 / \text{cat } 0.1 / \text{dog } 0.4$
 $VB \rightarrow \text{chased } 0.3 / \text{dodged } 0.5$
 $DT \rightarrow \text{the } 0.2 / a \quad 0.35$

Ans



The prob. of parse tree of the given i/p string is

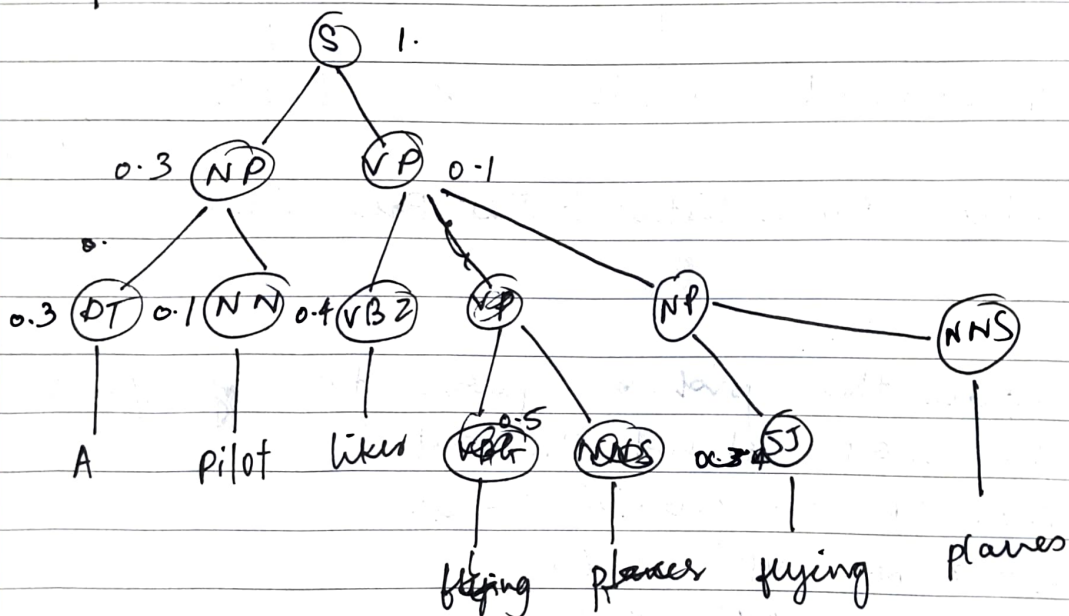
$$P = 0.75 \times 0.25 \times 0.35 \times 0.2 \times 0.25 \times 0.3 \times 0.25 \times 0.2 \times 0.1$$

$$= 0.00000492.$$

EX.

———. 'A pilot likes flying planes'.
q. in slide

Ans



$$P = 1 \times 0.3 \times 0.3 \times 0.3 \times 0.1 \times 0.1 \times 0.4 \times 0.4 \times 0.34 \times 0.34$$

$$= 0.00000612$$

Predictive Parser

Q: Parser: 'The dog chases a cat'

$S \rightarrow NP VP$

$NP \rightarrow \text{the } N / N$

$VP \rightarrow V NP$

$\text{Det} \rightarrow \text{the} / a$

$N \rightarrow \text{cat} / \text{dog}$

$V \rightarrow \text{chases} / \text{sees}$

Ans.

Stack (top $\rightarrow$ bottom)	i/p remaining	action
S \$	<u>the</u> dog chases a cat \$	table[S, 'the'] $\rightarrow S \rightarrow NP VP$ pop S, push NP VP
NP VP \$	the dog chases a cat \$	table[NP, 'the'] $\rightarrow NP \rightarrow \text{the } N$ pop NP, push N Det
Det N VP \$	the dog chases a cat \$	Table[Det, 'the'] $\rightarrow \text{Det} \rightarrow \text{the}$ Det is terminal $\rightarrow$ match (1st)
N VP \$	dog chases a cat \$	Table[N, 'dog'] $\rightarrow N \rightarrow \text{dog}$ match (2nd)
VP \$	chases a cat \$	table[VP, 'chases'] $\rightarrow VP \rightarrow V NP$ push NP V

o Early Parser (top-down)

④

Maximu

EX: 'Dog chases a cat'
 $S \rightarrow NP VP$
 $NP \rightarrow Det N/N$
 $VP \rightarrow V NP$
 $Det \rightarrow 'the' / 'a'$
 $N \rightarrow 'cat' / 'dog'$
 $V \rightarrow 'chases' / 'sees'$

<u>Ans</u>	<u>operation types</u>	<u>Production rules</u>	<u>Completed status</u>
	initialization	$S \rightarrow \cdot NP VP$	0
	predict	$NP \rightarrow \cdot Det N$	0
		$NP \rightarrow \cdot N$	
Scan: dog		$NP \rightarrow N \cdot$	1 (1st word is scanned)
complete		$S \rightarrow NP \cdot VP$	1
predict		$VP \rightarrow \cdot V NP$	1
Scan: chases		$VP \rightarrow V \cdot NP$	2 (2nd word is scanned)
predict		$NP \rightarrow \cdot Det N$	2
		$NP \rightarrow \cdot N$	2
Scan: the		$NP \rightarrow Det \cdot N$	3 (3rd word is scanned)
		$NP \rightarrow Det N \cdot$	
Scan: cat		$S \rightarrow NP VP \cdot$	4 (4th word is scanned)
complete			4